

File Systems

Definition

- A file is a named collection of related information that is recorded on secondary storage
- Text file is a sequence of characters organized into lines (and possibly pages).
- Source file is a sequence of functions, each of which is further organized as declarations followed by executable statements.
- Executable file is a series of code sections that the loader can bring into memory and execute

File Attributes

- Name
- Identifier
- Type
- Location
- Size
- Protection
- Timestamps and user identification

File Operations

- Creating
- Writing
- Reading
- Repositioning –seek
- Deleting
- Truncating

Information associated with open file

- File Pointer
- File-open Count
- Disk location
- Access rights

File Types

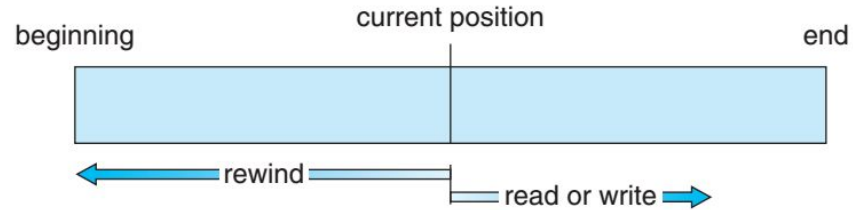
file type	usual extension	function
executable	exe, com, bin or none	ready-to-run machine-language program
object	obj, o	compiled, machine language, not linked
source code	c, cc, java, perl, asm	source code in various languages
batch	bat, sh	commands to the command interpreter
markup	xml, html, tex	textual data, documents
word processor	xml, rtf, docx	various word-processor formats
library	lib, a, so, dll	libraries of routines for programmers
print or view	gif, pdf, jpg	ASCII or binary file in a format for printing or viewing
archive	rar, zip, tar	related files grouped into one file, sometimes compressed, for archiving or storage
multimedia	mpeg, mov, mp3, mp4, avi	binary file containing audio or A/V information

Access Methods

Sequential Access: read next and write next

Direct Access: read n; write n

Indexed Access Method



sequential access	implementation for direct access
reset	<code>cp = 0;</code>
read_next	<code>read cp;</code> <code>cp = cp + 1;</code>
write_next	<code>write cp;</code> <code>cp = cp + 1;</code>

Directory and Disk Structure

Volume

One Disk

More than one disk

One part of disk

Directory Overview

Search for a file

Create a file

Delete a file

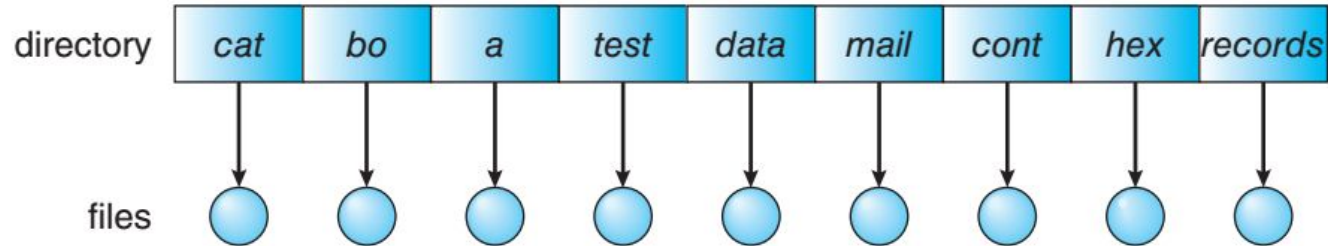
List a directory

Rename a file

Traverse a file system

Logical structure of directory

Single Level Directory

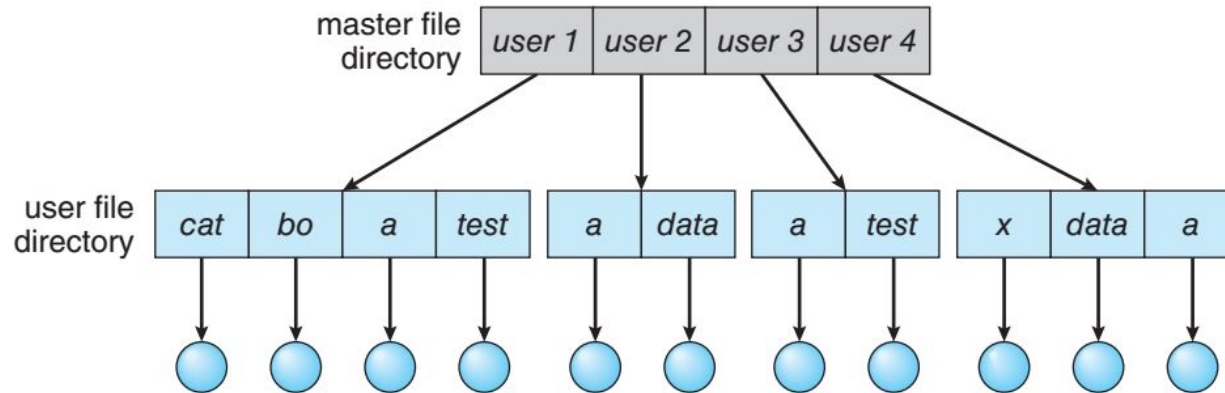


Two-Level Directory

MFD and UFD

Cooperation on some task

Path name



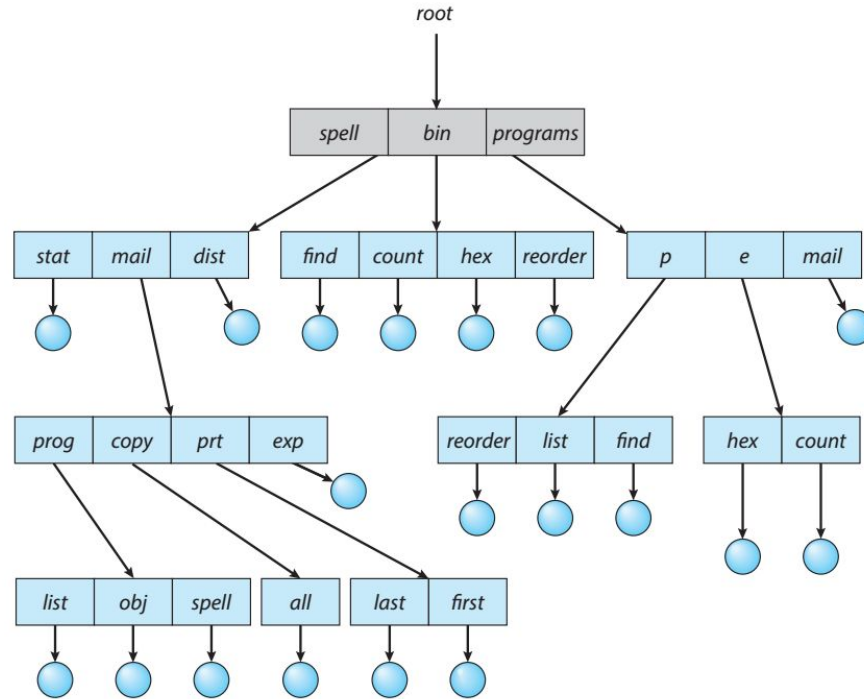
Tree Structure Directory

Current Directory

Relative and Absolute Path

Deletion of directory

Prohibits sharing of files



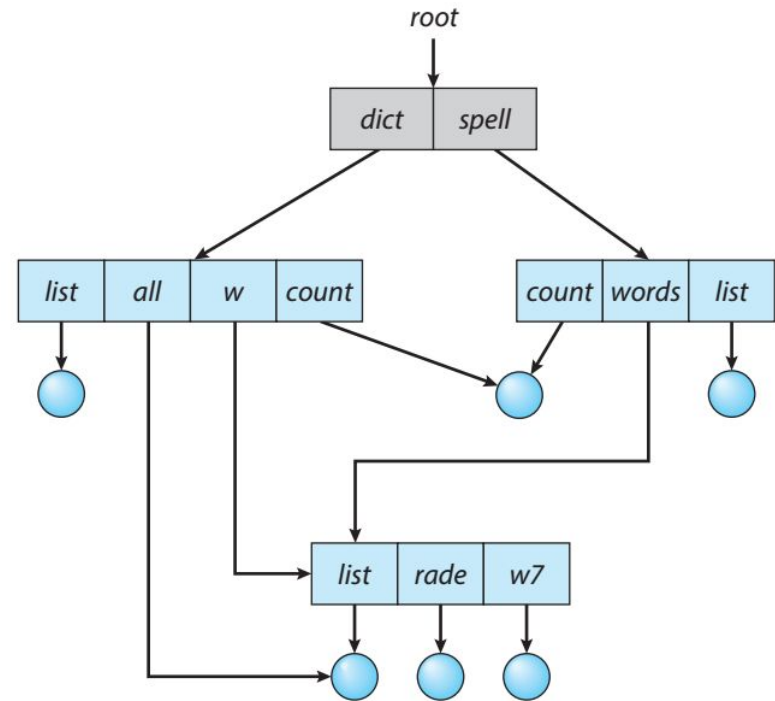
Acyclic Graph Directories

Allows sharing of subdirectories and files

Implementation : Links and shared files

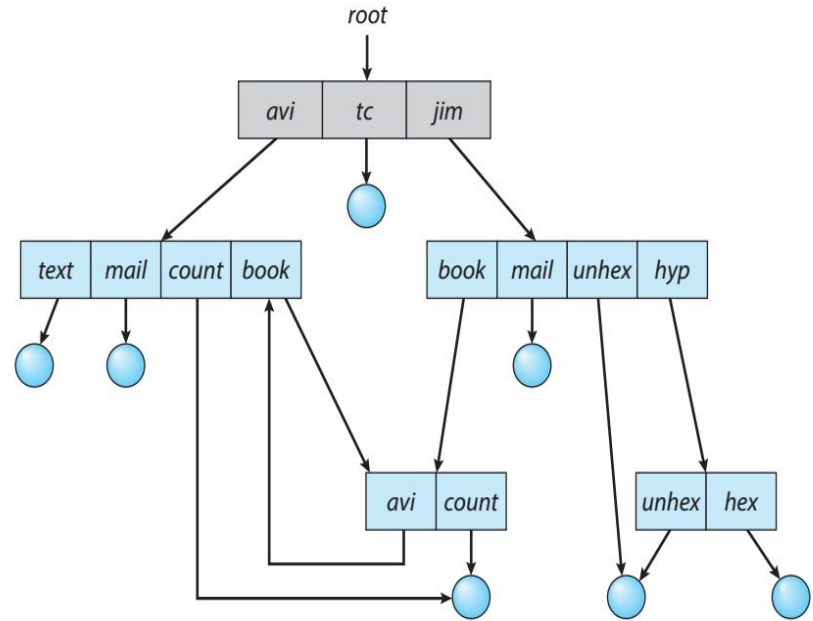
Problems:- Traversing, copy all the files to backup, Deletion (Dangling Pointer)

Solutions:- List of all the references, reference count

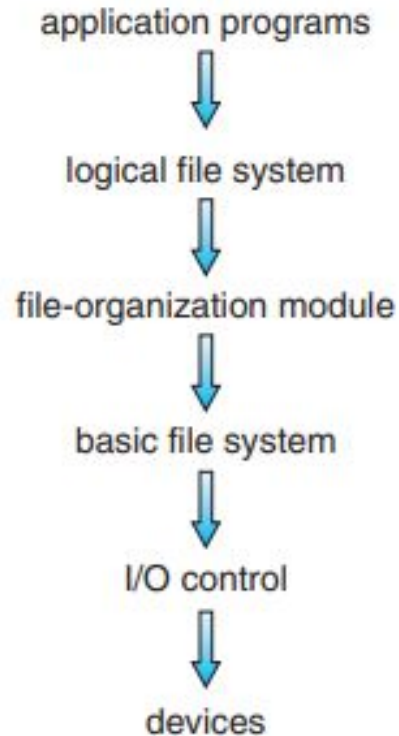


General Graph Directory

- Allows cycles
- Use Garbage collection
- Every time a new link is added, use a cycle detection algorithm to determine whether a cycle now exists.
- Disadvantage: Computationally expensive
- An alternative approach is to bypass links during directory traversal



File System Structure



File System Implementation

On-disk structure:-

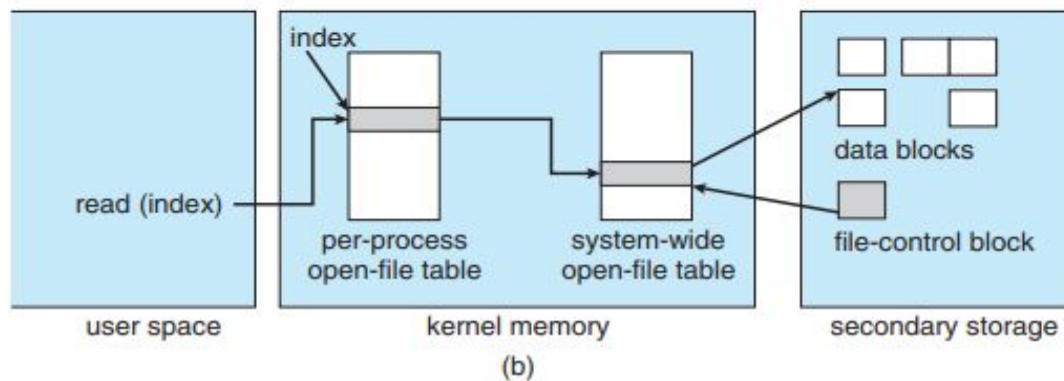
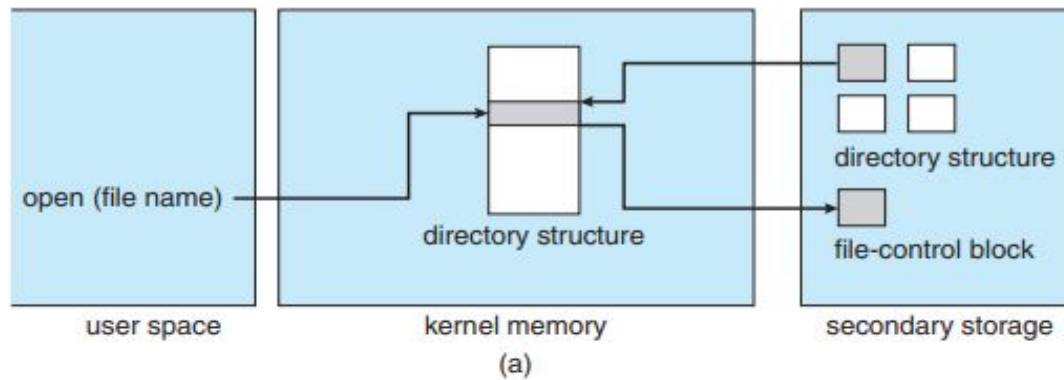
- Boot control block (Boot Block, Partition Boot Sector)
- Volume control block (Superblock, Master File Table)
- Directory structure (File names and associated inodes)
- FCB

In-memory structure:-

- In-memory mount table
- In-memory directory structure cache
- System-wide open-file table
- Per-process open-file table

FCB

file permissions
file dates (create, access, write)
file owner, group, ACL
file size
file data blocks or pointers to file data blocks



Directory Implementation

- Linear List
- Hash Table

Allocation Methods

Contiguous Allocation

Linked Allocation

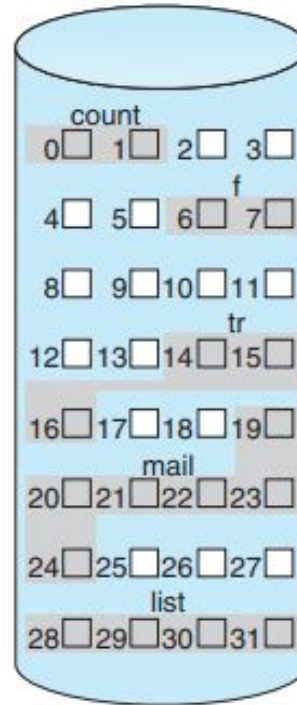
Indexed Allocation

Contiguous Allocation

Inflexible

Internal and External Fragmentation

Sequential and Random Access



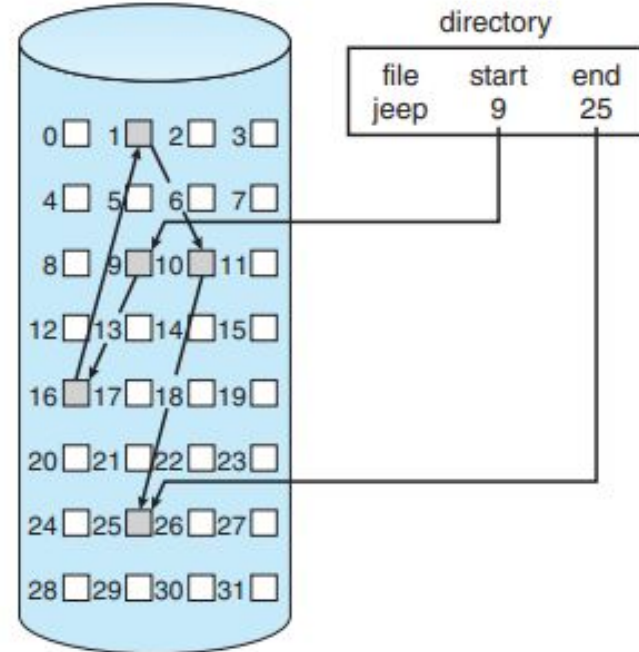
directory		
file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Linked Allocation

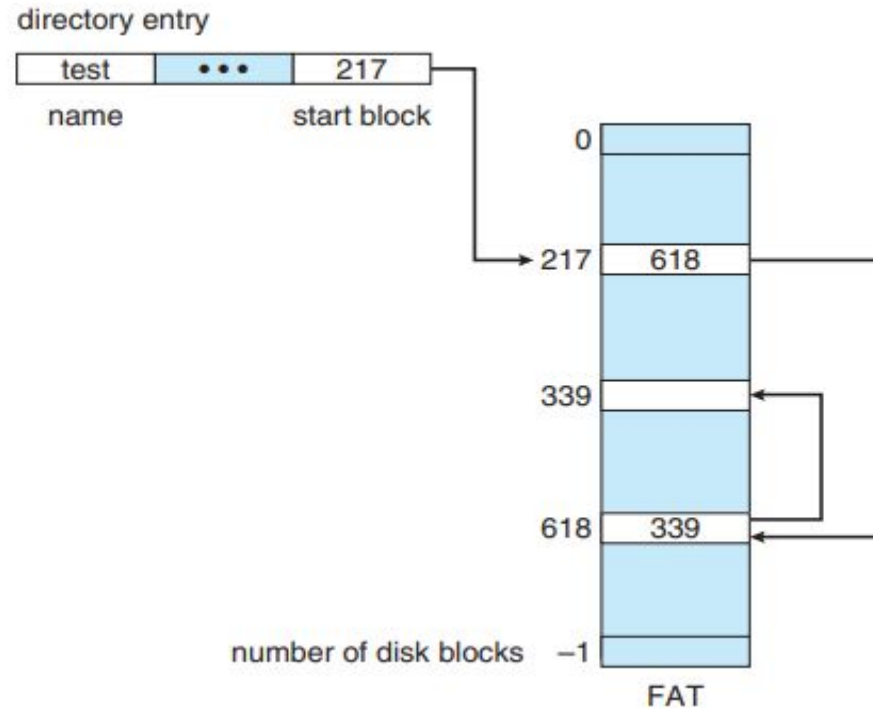
Flexible

Sequential Access

Internal Fragmentation



File Allocation Table (FAT)



Indexed Allocation

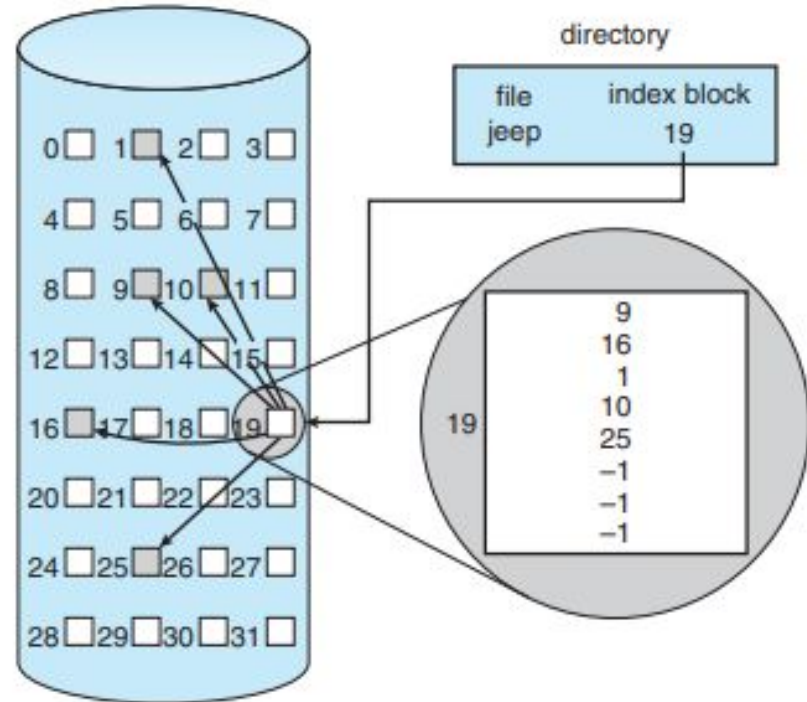
One Block is dedicated to indexing

Sequential and Random access

Internal Fragmentation

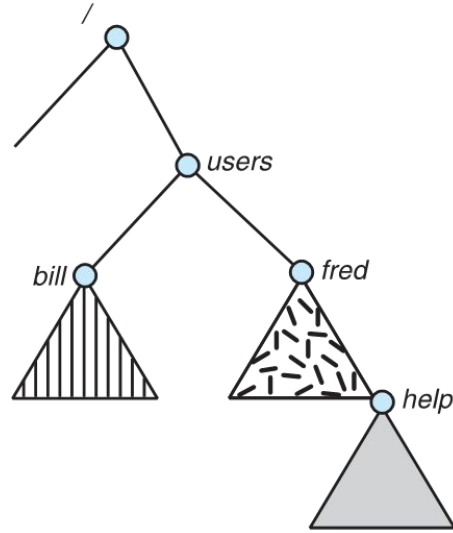
Flexible

Multi-level Indexing

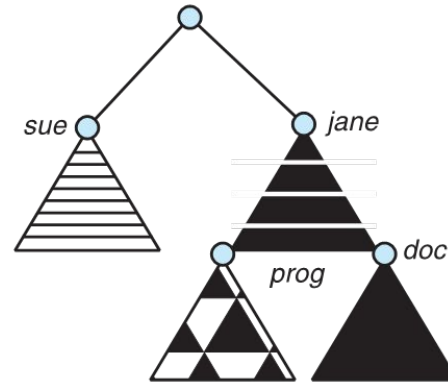


File System Mounting

- A file system has to be mounted before use.
- Attaching a file.



(a)



(b)

File Sharing

- Sharing of files on multi-user systems is desirable
- User IDs identify users, allowing permissions and protections to be per-user
- Group IDs allow users to be in groups, permitting group access rights
- Owner of a file / directory
- Group of a file / directory
- **Uses networking to allow file system access between systems**
- Manually via programs like FTP
- Using distributed file systems
- Semi automatically via the world wide web

Protection

- File owner/creator should be able to control:
- what can be done by whom
- Types of access
- Read
- Write
- Execute
- Append
- Delete
- List

Access Control

- Mode of access: read, write, execute
- Three classes of users on Unix / Linux

RWX

- a) owner access 7 1 1 1
- b) group access 6 1 1 0
- c) public access 1 0 0 1
- Ask manager to create a group (unique name), say G, and add some users to the group.
- For a particular file (say game) or subdirectory, define an appropriate access.
- Other Protection Approaches

Free Space Management

- File system maintains free-space list to track available blocks/clusters
- (Using term “block” for simplicity)
- Bit vector or bit map (n blocks)
- Bit map requires extra space
- Easy to get contiguous files
- Suitable for small disk but difficult to maintain for large disk

Linked List, Grouping and Counting

- Linked list (free list)
- Cannot get contiguous space easily
- No waste of space
- No need to traverse the entire list (if # free blocks recorded)
- Grouping
- Modify linked list to store address of next n-1 free blocks in first free block, plus a pointer to next block that contains free-block-pointers (like this one)
- Counting
- Because space is frequently contiguously used and freed, with contiguous-allocation allocation, extents, or clustering
- Keep address of first free block and count of following free blocks
- Free space list then has entries containing addresses and counts

Space Maps

- Used in ZFS
- Consider meta-data I/O on very large file systems
- Full data structures like bit maps couldn't fit in memory -> thousands of I/Os
- Divides device space into metaslab units and manages metaslabs
- Given volume can contain hundreds of metaslabs
- Each metaslab has associated space map
- Uses counting algorithm
- But records to log file rather than file system
- Log of all block activity, in time order, in counting format
- Metaslab activity -> load space map into memory in balanced-tree structure, indexed by offset
- Replay log into that structure
- Combine contiguous free blocks into single entry

Unix inode

